# Nodes	# Edges	Len. of Shortest Path	# Shortest Paths
23.0	36.0	3.8	1.6

Table 2 Statistics of the graph structure in ProsQA.

enforce that the new node cannot simultaneously serve as a descendant to both node 0 and 1. This separation maintains distinct families of nodes and balances their sizes to prevent model shortcuts.

After the graph is constructed, nodes without parents are assigned entity names, while other nodes receive concept names. To formulate a question of the form "Is [Entity] a [Concept A] or [Concept B]?", we designate node 0 in the graph as [Entity], a leaf node labeled 1 as [Concept A], and a leaf node labeled 2 as [Concept B]. This setup ensures a path from [Entity] to [Concept A] without any connection to [Concept B], introducing a moderately complex reasoning path. Finally, to avoid positional biases, [Concept A] and [Concept B] are randomly permuted in each question.

Algorithm 1 Graph Construction for ProsQA

```
edges ← {}
nodes ← {0, 1}
labels ← {0 : 1, 1 : 2}
                    ▷ Labels: 1 (descendant of node 0), 2 (descendant of node 1), 3 (both), 0 (neither).
groups ← {0 : {}, 1 : {0}, 2 : {1}, 3 : {}}
idx ← 2
while idx < N do
                    ▷ For each new node, randomly add edges from existing nodes
    n_in_nodes ← poisson(1.5)
    rand ← random()
    if rand ≤ 0.35 then
        candidates ← groups[0] ∪ groups[1]          ▷ Cannot be a descendant of node 1.
    else if rand ≤ 0.7 then
        candidates ← groups[0] ∪ groups[2]          ▷ Cannot be a descendant of node 0.
    else
        candidates ← nodes
    end if
    n_in_nodes ← min(len(candidates), n_in_nodes)
    weights ← [depth_to_root(c) · 1.5 + 1 ∀c ∈ candidates]
                    ▷ Define sampling weights to prioritize deeper nodes.
                    ▷ This way, the solution reasoning chain is expected to be longer.
    in_nodes ← random_choice(candidates, n_in_nodes, prob = weights/sum(weights))
    cur_label ← 0
    for in_idx ∈ in_nodes do
        cur_label ← cur_label | labels[in_idx]      ▷ Update label using bitwise OR.
        edges.append((in_idx, idx))
    end for
    groups[cur_label].append(idx)
    labels[idx] ← cur_label
    nodes ← nodes ∪ {idx}
    idx ← idx + 1
end while
```

A.3 Statistics

We show the size of all datasets in Table 3.

Dataset	Training	Validation	Test
GSM8k	385,620	500	1319
ProntoQA	9,000	200	800
ProsQA	17,886	300	500

Table 3 Statistics of the datasets.

B Clock-Time Reasoning Efficiency Metric

We present a clock-time comparison to evaluate reasoning efficiency. The reported values represent the average inference time per test case (in seconds), with a batch size of 1, measured on an Nvidia A100 GPU. For the no-CoT and CoT baselines, we employ the standard generate method from the `transformers`[3] library. Our results show that clock time is generally proportional to the number of newly generated tokens, as detailed in Table 1.

Method	GSM8k	ProntoQA	ProsQA
No-CoT	0.03	0.03	0.08
CoT	0.26	0.85	0.47
COCONUT	0.09	0.11	0.15

Table 4 Inference time (in seconds) comparison across tasks and methods.

C More Discussion

C.1 Using More Continuous Thoughts

In Figure 8 (II), we present the performance of COCONUT on GSM8k using $c \in \{0, 1, 2\}$. When experimenting with $c = 3$, we observe a slight performance drop accompanied by increased variance. Analysis of the training logs indicates that adding three continuous thoughts at once – particularly during the final stage transition – leads to a sharp spike in training loss, causing instability. Future work will explore finer-grained schedules, such as incrementally adding continuous thoughts one at a time while removing fewer language tokens, as in iCoT (Deng et al., 2024). Additionally, combining language and latent reasoning—e.g., generating the reasoning skeleton in language and completing the reasoning process in latent space—could provide a promising direction for improving performance and stability.

C.2 Coconut with Larger Models

We experimented with COCONUT on GSM8k using Llama 3.2-3B and Llama 3-8B (Dubey et al., 2024) with $c = 1$. We train them for 3 epochs in Stage 0, followed by 1 epoch per subsequent stage. The results are shown in Table 5.

Model	**no-CoT**	**Coconut (Ours)**
Llama 3.2-3B	26.0	31.7
Llama 3-8B	42.2	43.6

Table 5 Experimental results of applying COCONUT to larger Llama models. We report performance comparisons between models without CoT reasoning (no-CoT) and our proposed COCONUT method.

We observe consistent performance gains across both Llama 3.2-3B and Llama 3-8B models compared to the no-CoT baseline, though these improvements are not as pronounced as those previously demonstrated

[3] https://github.com/huggingface/transformers

with GPT-2. One possible reason is that larger models have already undergone extensive language-focused pre-training, making the transition to latent reasoning more challenging.

We emphasize that the primary goal of this paper is to highlight the promising attributes of latent-space reasoning and to initiate exploration in this new direction. Universally surpassing language-based CoT likely requires significant research efforts dedicated to **latent space pre-training**. We are encouraged by recent progress in this area (Geiping et al., 2025; Barrault et al., 2024; Gladstone et al., 2025). While these recent models provide scalable methods for latent representation learning, the latent spaces have not yet been explicitly optimized for reasoning. Integrating these recent advancements with COCONUT presents an exciting and promising avenue for future research.